

Introduction à l'algorithmique et la programmation

Algorithmique et structuration de données

Léna Gaubert

lenna.gaubert@proton.me

L3 TIM/TAL @ INALCO
Semestre 5

About me

Data Scientist @ [Arlequin AI](#)
Master pluriTAL (mention R&D) @ Sorbonne
Nouvelle
Mentorat @ Wild Code School & Centrale
Supélec
Contact: lana.gaubert@proton.me

- Séances de 3h : 1h de théorie et 2h de pratique
- Supports de cours (TP et diapos) en ligne sur mon site (<https://kittog.github.io/teachings>)
- 12 semaines de cours
- 2 partiels (mi et fin semestre) sur papier (& more?)

Apprentissage du langage Python

- connaître et manipuler différents types de données (entiers, chaînes, listes...)
- résoudre des problèmes algorithmiques
- déboguer un code Python
- produire du code lisible, clair, et *propre*

Quelques pré-requis

- vous avez accès à un ordinateur (Linux/Windows/MacOs)
- vous êtes familiarisé avec l'utilisation du terminal
- *vous voulez apprendre à programmer en Python*

Ce cours couvrira les sujets suivants :

- **Types de données** (*data types*) : entier, float, chaînes de caractères, listes, dictionnaires...
- **Opérations & instructions conditionnelles** (`if`, `else`)
- **Boucles** (`while`, `for`)
- **Fonctions**

Pourquoi (*encore*) apprendre à coder ?
What about vibe-coding?

Pourquoi Python, finalement ?

- **multiplateforme** (Linux/MacOs/Windows/Android...)
- langage de **haut niveau**, et **interprété** (pas besoin d'être compilé pour être exécuté)
- langage très **polyvalent**, utilisé dans de nombreux domaines (TAL, physique, biologie, finances, data science...)

Vérifier son installation Python (depuis le terminal)

- Dans le `shell Unix` (MacOs, Linux) ou `powershell` (Windows) :

```
python3 --version
```

Si vous n'avez pas Python sur votre machine : [How to install Python: a guide](#) (Real Python).

Créer un répertoire pour le cours

Toujours dans le terminal, depuis le répertoire HOME (pour y retourner : `cd ~`) : créer le répertoire `intro-python/scripts` .

```
mkdir intro-python/scripts
```

Vous pouvez utiliser l'éditeur de votre choix !

- [Visual Studio Code](#)
- [PyCharm](#)
- [Sublime Text](#)
- [Pulsar](#) (successeur d'[Atom](#), archivé depuis fin 2022)
- [Vim](#) (à vos risques et périls ; pour une alternative moderne voir [Helix](#))

Structure d'un script

Un script écrit en langage Python suit généralement la structure suivante :

1. **Imports de modules**
2. **Fonctions**
3. **Instructions** : Commandes exécutées par l'interpréteur Python (opérations, appel fonctions...)
4. **Commentaires** : Expliquent votre code en langage naturel. Précédé du caractère `#`, un commentaire est ignoré par l'interpréteur Python jusqu'à la fin de la ligne.

Hello world!

Dans l'éditeur de votre choix, ouvrez un fichier `hello_world.py` et écrivez les instructions suivantes.

```
# print message  
print("Hello, world!")
```

Pour exécuter le script :

```
python3 hello_world.py
```

Petite variation du script `hello_world.py`

```
# declare variables
first_name = input("first name:")
last_name = input("last name:")
# create message
message = "Hello " + first_name + "!"
# print message
print(message)
```

Fonctions `input()` et `print()`

`print()`

La fonction `print()` permet d'afficher des messages, des variables.

```
x = 4  
print("Le résultat est : ", x, end="!")
```

`input()`

La fonction `input()` permet de recevoir des données d'entrée de l'utilisateur, via la console.

```
x = input("Enter your name:")  
print("Hello", x, "!", sep="?", end="#")
```

Types de variables

Nom	Python	Description	Exemple
Integer	<code>int</code>	Nombre entier	<code>age=25</code>
Float	<code>float</code>	Nombre décimal	<code>temp=29.5</code>
String	<code>str</code>	Chaîne de caractères	<code>name="Eliza"</code>
Boolean	<code>bool</code>	Booléen. <code>True = 1 ;</code> <code>False = 0</code>	<code>is_here=True</code>

Types de variables

Nom	Python	Description	Exemple
List	<code>list</code>	Liste <i>ordonnée</i>	<code>l=[12, 14.5]</code>
Tuple	<code>tuple</code>	Liste <i>immutable</i>	<code>a=(2.0, 4.3)</code>
Dict	<code>dict</code>	Dictionnaire	<code>{"lang": "en"}</code>
Set	<code>set</code>	Ensemble	<code>{"en", "fr"}</code>
None	<code>NoneType</code>	Aucune valeur	<code>comment=None</code>

Catégories de variables

On peut regrouper les différents types de variables dans les catégories suivantes :

- *Text type* : `str`
- *Numeric types* : `int`, `float`
- *Sequence types* : `list`, `tuple`
- *Mapping type* : `dict`
- *Set type* : `set`
- *Boolean type*: `bool`
- *None type* : `None`

Casting : passer d'un type à un autre

```
# décimal -> entier  
x = 10.6 # float  
int_x = int(x)
```

```
# entier -> décimal  
y = 10  
float_y = float(y)
```

```
# nombre -> chaîne  
z = 42  
str_z = str(z)
```

```
# nombre -> booléen  
w = 0  
bool_w = bool(w)
```

Type d'une variable

Pour connaître le type d'une variable, utiliser la fonction `type()`.

```
age = 25
name = "Alan"
is_student = False

print(type(age))
print(type(name))
print(type(is_student))
```

Attribuer une valeur à une variable

En Python, il est possible de :

- attribuer des valeurs à plusieurs variables en une seule ligne

```
# three different students  
s1, s2, s3 = "Parry", "Tay", "Ada"
```

- donner la même valeur à plusieurs variables en une seule ligne

```
# three students with the same name  
s1 = s2 = s3 = "Eliza"
```

Opérations mathématiques

Opérateur	Dénomination	Effet
+	Addition	additionne
-	Soustraction	soustrait
*	Multiplication	multiplie
/	Division	divise
//	Division euclidienne	divise & arrondit
%	Quotient	de la division euclidienne
**	Puissance	

Opérations sur les types numériques

```
x = 45
y = x + 2 # 47
z = y - x # 2
p = z ** 2 # 4
d = x / 4 # 11.25
e = x // 4 # 11
r = x % 4 # 1
```

Opérations sur les chaînes de caractères

```
greeting = " Hello"  
greet_w_name = greeting + " Alan!"  
print(greet_w_name * 3)
```

Opérations invalides

```
>>> greeting ** 2  
>>> greeting + 4  
>>> greeting - "a"
```

PEP 8 – Style Guide for Python Code

Time to practice! \(.> ˘ <)/♡